PEMROGRAMAN BERBASIS FRAMEWORK
JOBSHEET 6

NAMA : Muhammad Fahreza Rohmansyah
KELAS : 3E
NO. ABSEN : 15
NIM : 2341720151

PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI MALANG
2026

Link Github:

https://github.com/rezfahreza/PBF_2026.git

Praktikum

2. Membuat API Produk

- Buat file pad pages/api/produk.js lalu tambahkan data statis

JS_7 > my-app > src > pages > api > produk.ts > handler > data

```typescript
import type { NextApiRequest, NextApiResponse } from 'next';

type Produk = {
    status: boolean;
    status_code: number;
    data:{
        id: string;
        nama: string;
        harga: number;
        ukuran: string;
        warna: string;
    }[];
};

export default function handler(
    req: NextApiRequest,
    res: NextApiResponse<Produk>
) {
    const data = [
        {
            id: '1',
            nama: 'Kaos Polos',
            harga: 10000,
            ukuran: 'L',
            warna: 'merah'
        },
        {
            id: '2',
            nama: 'Kaos Berlengan Panjang',
            harga: 15000,
            ukuran: 'M',
            warna: 'biru'
        }
    ];

    res.status(200).json({
        status: true,
        status_code: 200,
        data
    });
}
```

```
pages
  > about
  v api
    TS hello.ts        U
    TS produk.ts       U
```

- Akses: http://localhost:3000/api/produk

localhost:3000/api/produk

YouTube GitHub Wokwi - World

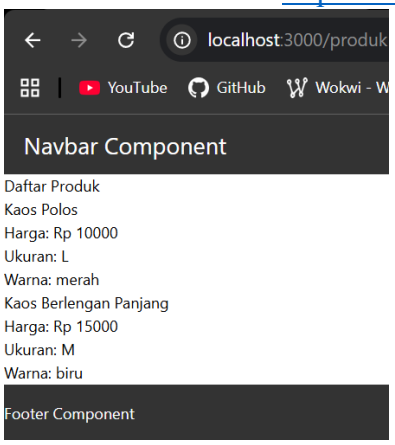

```
{
    "status": true,
    "status_code": 200,
    "data": [
        {
            "id": "1",
            "nama": "Kaos Polos",
            "harga": 10000,
            "ukuran": "L",
            "warna": "merah"
        },
        {
            "id": "2",
            "nama": "Kaos Berlengan Panjang",
            "harga": 15000,
            "ukuran": "M",
            "warna": "biru"
        }
    ]
}
```

3. Fetch Data API di Frontend

- Buka pages/product/index.tsx dan modifikasi kode

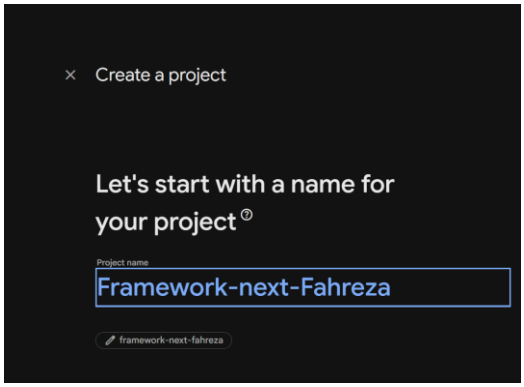
```
JS_7 > my-app > src > pages > produk > index.tsx > kategori
import { useRouter } from "next/router";
import { use, useEffect, useState } from "react";

type ProductType = {
    id: string;
    nama: string;
    harga: number;
    ukuran: string;
    warna: string;
};

const kategori = () => {

    const[products, setProducts] = useState<ProductType[]>([]);

    useEffect(() => {
        fetch("/api/produk")
            .then((responses) => responses.json())
            .then((responsesdata) => setProducts(responsesdata.data))
            .catch((error) => console.error("Error fetching products: ", error));
    }, []);

    return (
        <div>
            <h1>Daftar Produk</h1>
            {products.map((product: ProductType) => (
                <div key={product.id}>
                    <h2>{product.nama}</h2>
                    <p>Harga: Rp {product.harga}</p>
                    <p>Ukuran: {product.ukuran}</p>
                    <p>Warna: {product.warna}</p>
                </div>
            ))}
        </div>
    );
};

export default kategori;
```

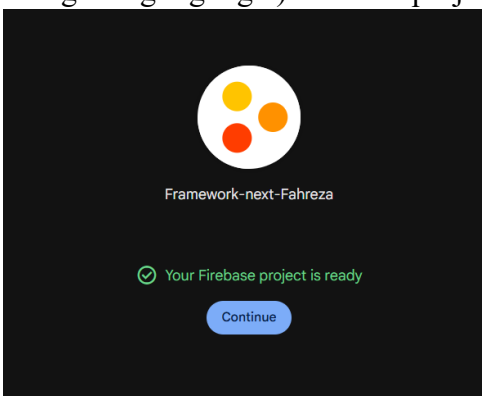
- Jalankan browser http://localhost:3000/produk



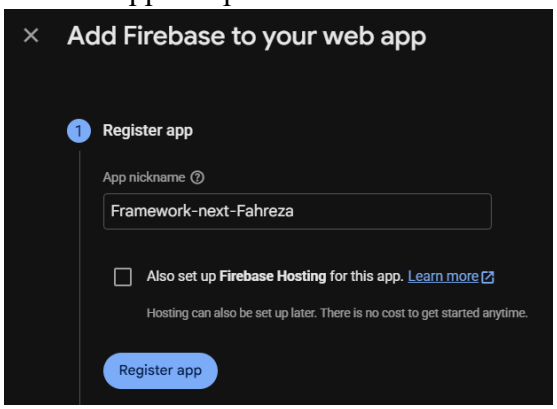
5. Setup Firebase

- Buka Firebase Go To Console (login dengan login google) dan buat projek baru

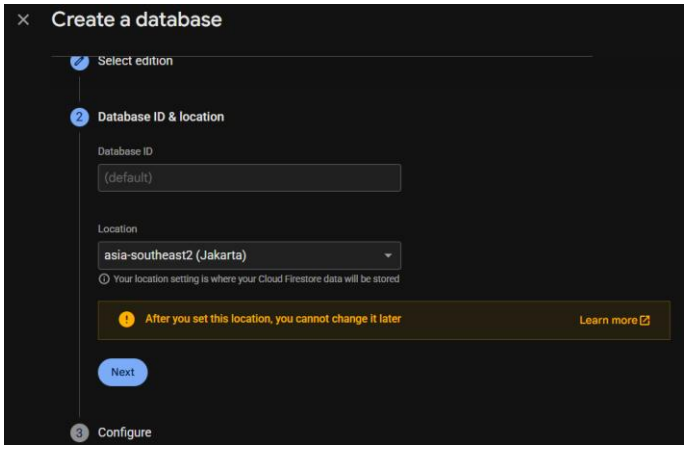




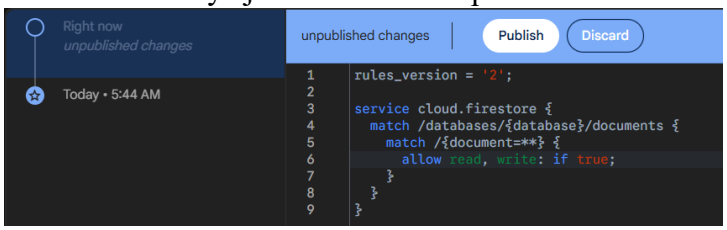
- Add app dan pilih web



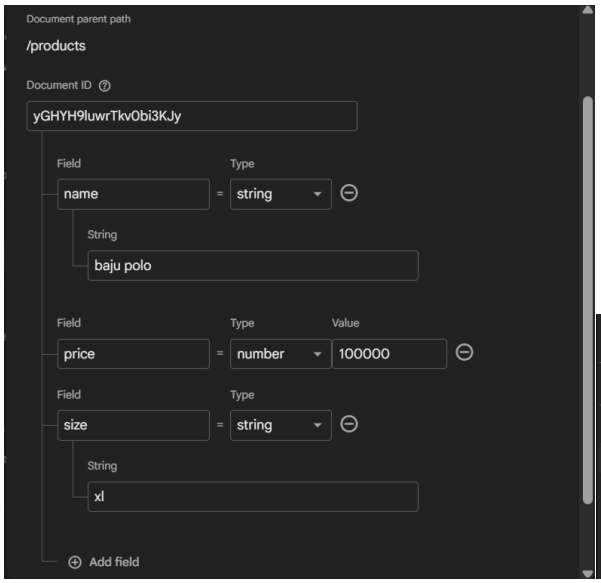
- Aktifkan Firestore Database

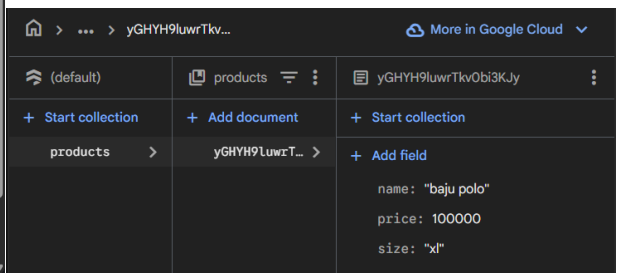


- Ubah rulesnya jadi true dan klik publish



- Buat collection:





6. Install Firebase

- npm install firebase

```
PS E:\PBF_2026\JS_7\my-app> npm install firebase

added 79 packages, and audited 468 packages in 2m

154 packages are looking for funding
  run `npm fund` for details
```

- Buat folder dan file ts pada pages utlis/db/firebase.ts

```
utils\db
TS firebase.ts    U
```

- Copy paste yang ada pada kotak merah ke file firebase.ts

7. Konfigurasi Environment Variable agar credensial firebase tidak dapat dilihat saat dipush di repository

- Buat file: .env.local dan memodifikasinya untuk mengisikan konfigurasi yang ada pada firebase

```
JS_7 > my-app > .env.local
FIREBASE_API_KEY=AIzaSyANjLS
FIREBASE_AUTH_DOMAIN=framewo
FIREBASE_PROJECT_ID=framewor
FIREBASE_STORAGE_BUCKET=fram
FIREBASE_MESSAGING_SENDER_ID
FIREBASE_APP_ID=1:7536894553
```

8. Konfigurasi Firebase

- Modifikasi firebase.ts

```
JS_7 > my-app > src > pages > utils > db > TS firebase.ts > default
// Import the functions you need from the SDKs you need
import { initializeApp } from "firebase/app";
import { getAnalytics } from "firebase/analytics";
// TODO: Add SDKs for Firebase products that you want to use
// https://firebase.google.com/docs/web/setup#available-libraries

// Your web app's Firebase configuration
// For Firebase JS SDK v7.20.0 and later, measurementId is optional
const firebaseConfig = {
  apiKey: process.env.FIREBASE_API_KEY,
  authDomain: process.env.FIREBASE_AUTH_DOMAIN,
  projectId: process.env.FIREBASE_PROJECT_ID,
  storageBucket: process.env.FIREBASE_STORAGE_BUCKET,
  messagingSenderId: process.env.FIREBASE_MESSAGING_SENDER_ID,
  appId: process.env.FIREBASE_APP_ID,
};

// Initialize Firebase
const app = initializeApp(firebaseConfig);
const analytics = getAnalytics(app);

export default app;
```

9. Ambil Data dari Firestore

- Buat file ulits/db/servicefirebase.js dan modifikasi

```
S_7 > my-app > src > pages > utils > db > TS servicefirebase.ts > retrieveProducts
import { getFirestore, collection, getDocs } from "firebase/firestore";
import app from "./firebase";

const db = getFirestore(app);

export async function retrieveProducts(collectionName: string) {
    const snapshot = await getDocs(collection(db, collectionName));
    const data = snapshot.docs.map((doc) => ({
        id: doc.id,
        ...doc.data(),
    }));
    return data;
}
```

10. API Mengambil Data Firebase

- Edit pages/api/product.js

```
JS_7 > my-app > src > pages > api > TS produk.ts > handler
import type { NextApiRequest, NextApiResponse } from 'next';
import { retrieveProducts } from '../utils/db/servicefirebase';

type Data = {
    status: boolean;
    status_code: number;
    data: any;
};

export default async function handler(
    req: NextApiRequest,
    res: NextApiResponse<Data>
) {
    const data = await retrieveProducts("products");

    res.status(200).json({
        status: true,
        status_code: 200,
        data
    });
}
```

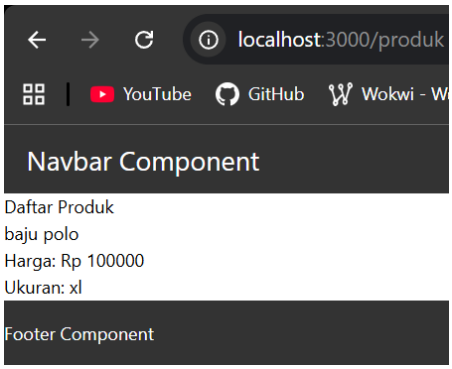
- Jalankan browser http://localhost:3000/api/produk

```
localhost:3000/api/produk
YouTube  GitHub  Wokwi - World'

{
    "status": true,
    "status_code": 200,
    "data": [
        {
            "id": "yGHYH9luwrTkv0bi3KJy",
            "size": "xl",
            "name": "baju polo",
            "price": 100000
        }
    ]
}
```

- Modifikasi index.ts pada produk sesuaikan nama typenya dan db nya

```
JS_7 > my-app > src > pages > produk > index.tsx > kategori > products.map() callback
import { useRouter } from "next/router";
import { use, useEffect, useState } from "react";

type ProductType = {
    id: string;
    name: string;
    price: number;
    size: string;
};

const kategori = () => {

    const[products, setProducts] = useState([]);

    useEffect(() => {
        fetch("/api/produk")
            .then((responses) => responses.json())
            .then((responsesdata) => setProducts(responsesdata.data))
            .catch((error) => console.error("Error fetching products: ", error));
    }, []);

    return (
        <div>
            <h1>Daftar Produk</h1>
            {products.map((products: ProductType) => (
                <div key={products.id}>
                    <h2>{products.name}</h2>
                    <p>Harga: Rp {products.price}</p>
                    <p>Ukuran: {products.size}</p>
                </div>
            ))}
        </div>
    );
};

export default kategori;
```

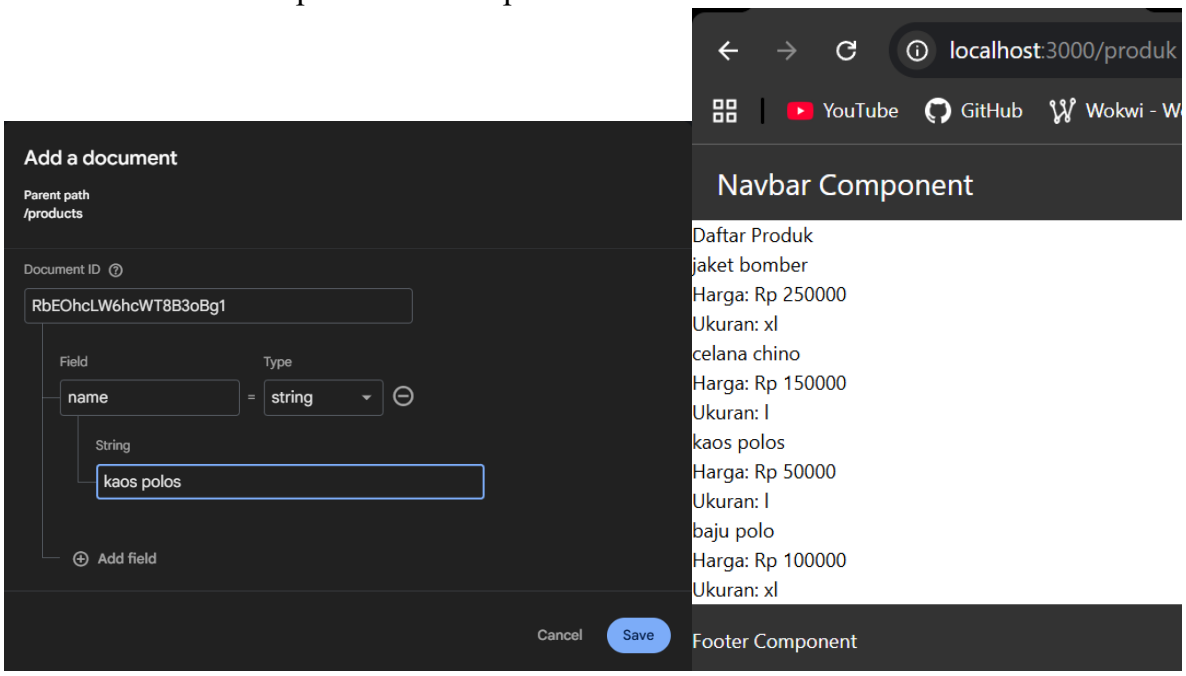
- Jalankan browser



Tugas Praktikum

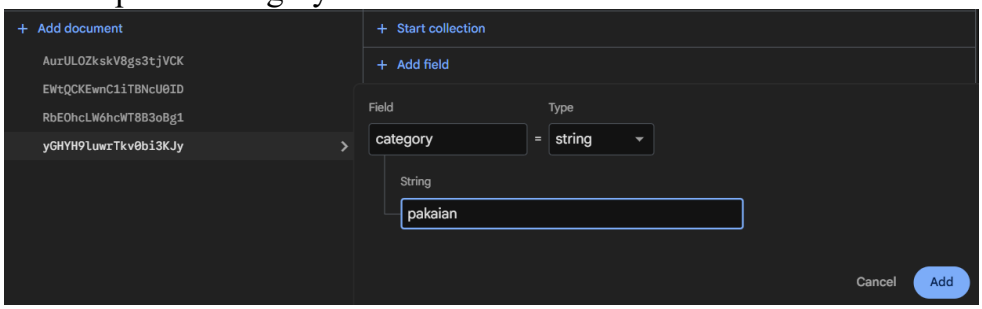
Tugas 1 (Wajib)

- Tambahkan minimal 3 data produk di Firestore
- Pastikan data tampil di halaman produk

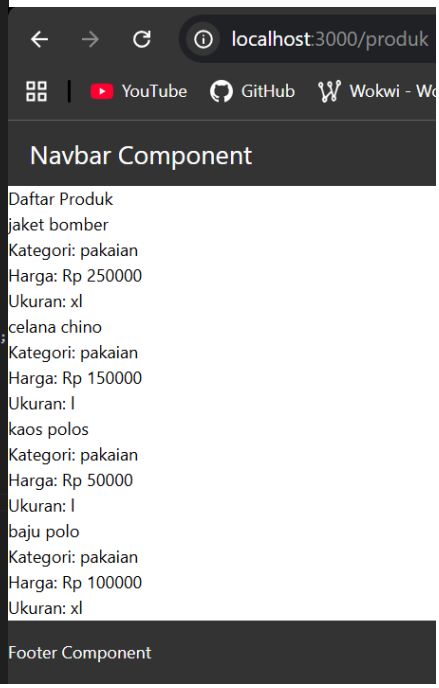


Tugas 2 (Wajib)

- Tambahkan field baru:
  - category
- Tampilkan category di frontend



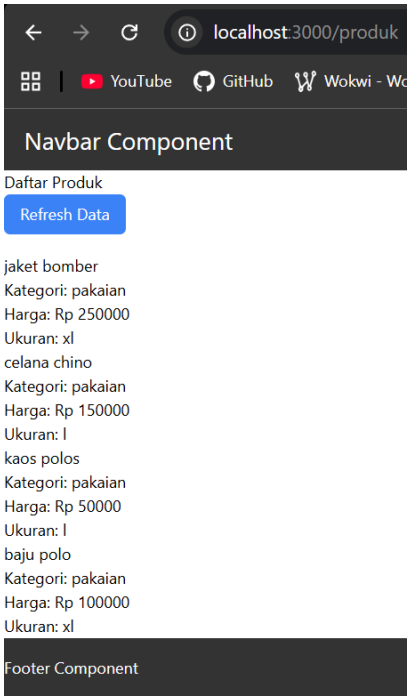
```
JS_7 > my-app > src > pages > produk > index.tsx > kategori > products.map() callback
import { useRouter } from "next/router";
import { use, useEffect, useState } from "react";

type ProductType = {
    id: string;
    name: string;
    price: number;
    size: string;
    category: string;
};

const kategori = () => {

    const[products, setProducts] = useState([]);

    useEffect(() => {
        fetch("/api/produk")
            .then((responses) => responses.json())
            .then((responsesdata) => setProducts(responsesdata.data))
            .catch((error) => console.error("Error fetching products: ", error));
    }, []);

    return (
        <div>
            <h1>Daftar Produk</h1>
            {products.map((products: ProductType) => (
                <div key={products.id}>
                    <h2>{products.name}</h2>
                    <p>Kategori: {products.category}</p>
                    <p>Harga: Rp {products.price}</p>
                    <p>Ukuran: {products.size}</p>
                </div>
            ))}
        </div>
    );
};

export default kategori;
```



Tugas 3 (Pengayaan)

- Tambahkan tombol Refresh Data
- Gunakan fetch ulang tanpa reload halaman

```
const fetchProducts = () => {
    fetch("/api/produk")
        .then((response) => response.json())
        .then((responseData) => setProducts(responseData.data))
        .catch((error) => console.error("Error fetching products:", error));
};

useEffect(() => {
    fetchProducts();
}, []);

return (
    <div>
        <h1>Daftar Produk</h1>
        <button className="bg-blue-500 text-white px-4 py-2 rounded-md mb-5 hover:bg-blue-600 transition" onClick={fetchProducts}>
            Refresh Data
        </button>
        {products.map((products: ProductType) => (
            <div key={products.id}>
                <h2>{products.name}</h2>
                <p>Kategori: {products.category}</p>
                <p>Harga: Rp {products.price}</p>
                <p>Ukuran: {products.size}</p>
            </div>
        ))}
    </div>
);
};
```



Pertanyaan Refleksi

1. Apa fungsi API Routes pada Next.js?
   Jawab: Memungkinkan pembuatan backend (API) langsung di dalam Next.js tanpa server tambahan. File di pages/api/ otomatis menjadi endpoint yang mengembalikan data JSON
2. Mengapa .env.local tidak boleh di-push ke repository?
   Jawab: Karena file tersebut berisi informasi sensitif seperti API Key, database credentials, atau rahasia dapur aplikasi yang jika tersebar bisa membahayakan keamanan data
3. Apa perbedaan data statis dan data dinamis?
   Jawab:
   - Data Statis: Data tetap yang tidak berubah (contoh: teks di halaman Tentang Kami).
   - Data Dinamis: Data yang berubah-ubah karena diambil dari database (contoh: daftar produk dari Firebase yang bisa ditambah/hapus).
4. Mengapa Next.js disebut framework fullstack?
   Jawab: Karena Next.js bisa menangani sisi Frontend (tampilan UI menggunakan React) sekaligus sisi Backend (API Routes dan akses database) dalam satu proyek yang sama